```
#set1
#random forest
#ff
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Dataset Setup
data = {
    'Outlook': ['sunny','sunny','overcast','rainy','rainy','rainy',
                'overcast','sunny','sunny','rainy','sunny','overcast',
                'overcast','rainy'],
    'Temp': ['hot','hot','hot','mild','cool','cool',
             'cool','mild','cool','mild','mild','mild',
             'hot','mild'],
    'Humidity': ['high','high','high','high','normal','normal',
                 'normal','high','normal','normal','normal',
                 'high','normal','high'],
    'Wind': ['weak','strong','weak','weak','weak','strong',
             'strong','weak','weak','weak','strong',
             'strong','weak','strong'],
    'Play Tennis': ['no','no','yes','yes','yes','no',
```

```
                'yes','no','yes','yes','yes',
                'yes','yes','no']
}
df = pd.DataFrame(data)

# 2. Encoding Categorical Data
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# 3. Splitting Features and Target
x = df.drop('Play Tennis', axis=1)
y = df['Play Tennis']
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=10)

# 4. Model Training
rf = RandomForestClassifier(n_estimators=10, criterion="gini", random_state=100)
rf.fit(x_train, y_train)

# 5. Predictions and Evaluation
y_pred = rf.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
c_mat = confusion_matrix(y_test, y_pred)
c_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:")
```

```
print(c_mat)
print("\nClassification Report:")
print(c_rep)

# 6. Feature Importance Plot (FIXED WARNING HERE)
plt.figure(figsize=(8, 5))
sns.barplot(
    x=rf.feature_importances_,
    y=x.columns,
    hue=x.columns,     # Fixes the palette warning
    palette="viridis",
    legend=False       # Removes redundant legend
)
plt.title("Feature Importance")
plt.show()

# 7. Visualizing one of the Decision Trees from the Forest
plt.figure(figsize=(19,7))
plot_tree(
    rf.estimators_[0],
    filled=True,
    feature_names=list(x.columns),
    class_names=["no", "yes"],
    rounded=True
)
plt.title("Visualizing Tree 0 from Random Forest")
plt.show()
```

```
#linearregression

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data[:, 2].reshape(-1, 1)
y = iris.data[:, 3]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Slope (m):", model.coef_[0])
print("Intercept (b):", model.intercept_)

plt.figure(figsize=(8, 5))
plt.scatter(X_train, y_train, color='blue', alpha=0.5, label='Training Data')
plt.scatter(X_test, y_test, color='green', alpha=0.8, label='Testing Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Linear Regression Line')

plt.xlabel("Petal Length (cm)")
```

```
plt.ylabel("Petal Width (cm)")
plt.title("Linear Regression: Petal Length vs. Petal Width (Iris Dataset)")
plt.legend()
plt.show()
```

#set2 a)rf on iris

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

x = df.drop('target', axis=1)
y = df['target']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=10)
```

```
rf = RandomForestClassifier(n_estimators=10, criterion="gini", random_state=100)
rf.fit(x_train, y_train)

y_pred = rf.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
c_mat = confusion_matrix(y_test, y_pred)
c_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:\n", c_mat)
print("\nClassification Report:\n", c_rep)

plt.figure(figsize=(8,5))
sns.barplot(
   x=rf.feature_importances_,
   y=x.columns,
   hue=x.columns,
   palette="viridis",
   legend=False
)
plt.title("Feature Importance (Iris Dataset)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.show()

plt.figure(figsize=(19,7))
```

```
plot_tree(
    rf.estimators_[0],
    filled=True,
    feature_names=list(x.columns),
    class_names=list(iris.target_names),
    rounded=True
)
plt.title("Visualization of Tree 0 in the Random Forest")
plt.show()

#2b kmeans

import numpy as np
import matplotlib.pyplot as plt

X = np.array([2, 4, 10, 12, 3, 20, 30, 11, 25], dtype=float)

k = 2

centroids = np.array([4, 11], dtype=float)

def plot_clusters(X, labels, centroids, iteration):
    plt.figure()

    for i in range(k):
        points = X[labels == i]
        plt.scatter(points, np.zeros_like(points), label=f"Cluster {i}")
```

```
    plt.scatter(centroids, np.zeros_like(centroids),
                marker='X', s=200, label='Centroids')

    plt.yticks([])
    plt.title(f"Iteration {iteration}")
    plt.legend()
    plt.show()

for iteration in range(1, 6):
    print(f"\n========== Iteration {iteration} ==========")

    distances = np.abs(X[:, np.newaxis] - centroids)

    labels = np.argmin(distances, axis=1)

    print("Point\tDist to C0\tDist to C1\tCluster")
    for i in range(len(X)):
        print(f"{X[i]:.1f}\t{distances[i][0]:.2f}\t\t{distances[i][1]:.2f}\t\t{labels[i]}")

    new_centroids = np.array([
        X[labels == i].mean() for i in range(k)
    ])

    print("\nOld Centroids:", centroids)
    print("New Centroids:", new_centroids)
    if np.allclose(centroids, new_centroids):
        print("\n Done!")
        break
```

```
#3a bck prp
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([[0],
              [1],
              [1],
              [0]])
input_neurons = 2
hidden_neurons = 2
output_neurons = 1
np.random.seed(0)
W1 = np.random.rand(input_neurons, hidden_neurons)
b1 = np.random.rand(1, hidden_neurons)

W2 = np.random.rand(hidden_neurons, output_neurons)
b2 = np.random.rand(1, output_neurons)
learning_rate = 0.1
epochs = 10000
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + b1
```

```
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)
    hidden_error = np.dot(d_output, W2.T)
    d_hidden = hidden_error * sigmoid_derivative(hidden_output)
    W2 += np.dot(hidden_output.T, d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += np.dot(X.T, d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

print("Final Output:")
print(final_output)
binary_output = (final_output > 0.5).astype(int)
print("Binary Output:")
print(binary_output)

#3b kmeans
#2b kmeans

import numpy as np
import matplotlib.pyplot as plt

X = np.array([2, 4, 10, 12, 3, 20, 30, 11, 25], dtype=float)

k = 2
```

```
centroids = np.array([4, 11], dtype=float)

def plot_clusters(X, labels, centroids, iteration):
    plt.figure()

    for i in range(k):
        points = X[labels == i]
        plt.scatter(points, np.zeros_like(points), label=f"Cluster {i}")

    plt.scatter(centroids, np.zeros_like(centroids),
                marker='X', s=200, label='Centroids')

    plt.yticks([])
    plt.title(f"Iteration {iteration}")
    plt.legend()
    plt.show()

for iteration in range(1, 6):
    print(f"\n========== Iteration {iteration} ==========")

    distances = np.abs(X[:, np.newaxis] - centroids)

    labels = np.argmin(distances, axis=1)

    print("Point\tDist to C0\tDist to C1\tCluster")
    for i in range(len(X)):
        print(f"{X[i]:.1f}\t{distances[i][0]:.2f}\t\t{distances[i][1]:.2f}\t\t{labels[i]}")
```

```
    new_centroids = np.array([
        X[labels == i].mean() for i in range(k)
    ])

    print("\nOld Centroids:", centroids)
    print("New Centroids:", new_centroids)

    if np.allclose(centroids, new_centroids):
        print("\n Done!")
        break

    centroids = new_centroids
plot_clusters(X, labels, centroids, iteration)
new_point = 19

distances = np.abs(new_point - centroids)
cluster = np.argmin(distances)

print("New Point:", new_point)
print("Distances:", distances)
print("Predicted Cluster:", cluster)
```

```
#set4a naive bayes

import numpy as np

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB


train_texts = ["Win money now", "Claim free prize", "Hello how are you", "Lets meet
tomorrow"]

train_labels = [1, 1, 0, 0]


test_texts = ["Earn money fast", "Good morning let us meet"]


vectorizer = CountVectorizer()

X_train = vectorizer.fit_transform(train_texts)

X_test = vectorizer.transform(test_texts)


model = MultinomialNB()

model.fit(X_train, train_labels)


y_pred = model.predict(X_test)


print("Predictions:", y_pred)



#4b

#linearregression


import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data[:, 2].reshape(-1, 1)
y = iris.data[:, 3]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Slope (m):", model.coef_[0])
print("Intercept (b):", model.intercept_)

plt.figure(figsize=(8, 5))
plt.scatter(X_train, y_train, color='blue', alpha=0.5, label='Training Data')
plt.scatter(X_test, y_test, color='green', alpha=0.8, label='Testing Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Linear Regression Line')

plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Linear Regression: Petal Length vs. Petal Width (Iris Dataset)")
plt.legend()
plt.show()
```

#set 5

#Write a Python program using NumPy to create a 1D array of 10 elements. Demonstrate indexing, slicing, and modification of specific elements.

```
import numpy as np


arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])


print("Original Array:")


print(arr)


print("\nIndexing Examples:")


print("Element at index 0:", arr[0])


print("Element at index 4:", arr[4])


print("Last element:", arr[-1])


print("\nSlicing Examples:")


print("Elements from index 2 to 5:", arr[2:6])


print("First 5 elements:", arr[:5])


print("Elements from index 5 to end:", arr[5:])


print("Every 2nd element:", arr[::2])
```

```
arr[0] = 999          # Modify first element

arr[3:6] = [111,222,333]  # Modify multiple elements using slicing

print("\nModified Array:")

print(arr)



#5b
#Dataset: X = [[2.5, 2.4], [1.5, 1.9], [3.5, 3.0], [2.0, 2.1]]
import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

X = np.array([[2.5,2.4],

          [1.5,1.9],

          [3.5,3.0],

          [2.0,2.1]])

pca = PCA(n_components=1)
```

```
X_reduced = pca.fit_transform(X)

print("Projected Data:\n", X_reduced)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)

plt.scatter(X[:,0], X[:,1])

plt.title("Original 2D Data")

plt.subplot(1,2,2)

plt.scatter(X_reduced, [0]*len(X_reduced))

plt.title("Projected 1D Data")

plt.yticks([])

plt.tight_layout()

plt.show()
```

```
#set6 knn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler

data = pd.DataFrame({
    "Name": ["Ajay", "Mark", "Sara", "Faiza", "Sachin",
             "Rahul", "Pooja", "Smith", "Lakshmi", "Michael"],
    "Age": [32, 40, 16, 34, 34, 40, 20, 15, 55, 15],
    "Gender": ["M", "M", "F", "F", "M",
               "M", "F", "M", "F", "M"],
    "Sport": ["Football", "Neither", "Cricket", "Cricket", "Neither",
              "Cricket", "Neither", "Football", "Football", "Football"]
})

gender_encoder = LabelEncoder()
data["Gender_encoded"] = gender_encoder.fit_transform(data["Gender"])
sport_encoder = LabelEncoder()
data["Sport_encoded"] = sport_encoder.fit_transform(data["Sport"])

X = data[["Age", "Gender_encoded"]].values
y = data["Sport_encoded"].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
k = 3
```

```
model = KNeighborsClassifier(n_neighbors=k)
model.fit(X_scaled, y)
test_age = 5
test_gender = "F"
test_gender_encoded = gender_encoder.transform([test_gender])

X_test = np.array([[test_age, test_gender_encoded[0]]])
X_test_scaled = scaler.transform(X_test)

prediction = model.predict(X_test_scaled)
predicted_sport = sport_encoder.inverse_transform(prediction)
print("Predicted Sport for Angelina:", predicted_sport[0])

plt.figure(figsize=(8,6))
for sport in np.unique(y):
    mask = (y == sport)
    plt.scatter(X[mask, 0], X[mask, 1],
                label=sport_encoder.inverse_transform([sport])[0],
                s=100)

plt.scatter(test_age, test_gender_encoded[0],
            marker='*', color='black', s=200,
            label=f"Angelina (Predicted: {predicted_sport[0]})")
plt.xlabel("Age")
plt.ylabel("Gender (0=F, 1=M)")
plt.title("KNN Classification of Sports by Age and Gender")
plt.legend()
plt.show()
```

```
#6b lr

#linearregression


import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split


iris = load_iris()

X = iris.data[:, 2].reshape(-1, 1)

y = iris.data[:, 3]


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = LinearRegression()

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print("Slope (m):", model.coef_[0])

print("Intercept (b):", model.intercept_)


plt.figure(figsize=(8, 5))

plt.scatter(X_train, y_train, color='blue', alpha=0.5, label='Training Data')

plt.scatter(X_test, y_test, color='green', alpha=0.8, label='Testing Data')

plt.plot(X_test, y_pred, color='red', linewidth=2, label='Linear Regression Line')
```

```
plt.xlabel("Petal Length (cm)")

plt.ylabel("Petal Width (cm)")

plt.title("Linear Regression: Petal Length vs. Petal Width (Iris Dataset)")

plt.legend()

plt.show()


#set7 a

#Write a Python program using Matplotlib to draw histograms for each numerical feature and a single boxplot for all features.

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris


import pandas as pd


iris = load_iris()


df = pd.DataFrame(iris.data, columns=iris.feature_names)


df.hist()


plt.show()


plt.boxplot(df.values)


plt.show()
```

```
#7B IRIS PCA
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. Load Dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Standardize Features
X_scaled = StandardScaler().fit_transform(X)

# 3. Apply PCA (Reduce to 2 Dimensions)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# 4. Visualize the 2D Projected Data
plt.figure(figsize=(7, 5))
for i, name in enumerate(iris.target_names):
    plt.scatter(
        X_pca[y == i, 0], X_pca[y == i, 1], label=name, alpha=0.8, edgecolors="k"
    )
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on Iris Dataset")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```

#set 8 3x3 Write a Python program using NumPy to create a 3×3 array and extract the first two

```
import numpy as np

arr = np.array([[1, 2, 3],

        [4, 5, 6],

        [7, 8, 9]])

print("Original 3x3 Array:")

print(arr)

first_two_rows = arr[0:2, :]

print("\nFirst Two Rows:")

print(first_two_rows)
first_two_columns = arr[:, 0:2]
print("\nFirst Two Columns:")
print(first_two_columns)
last_row = arr[-1, :]
print("\nLast Row:")
print(last_row
```

```
#8B lda syntethis data two class

import numpy as np

import matplotlib.pyplot as plt

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2,

                    n_classes=2, n_redundant=0,

                    random_state=0)

lda = LinearDiscriminantAnalysis(n_components=1)

X_lda = lda.fit_transform(X, y)
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Original Data")
plt.subplot(1,2,2)
plt.scatter(X_lda, [0]*len(X_lda), c=y)
plt.title("Projected Data (LDA)")
plt.yticks([])
plt.tight_layout()
plt.show()
```

#SET 9A

#3. a) Write a Python program using Matplotlib to draw histograms for each numerical feature

#and a single boxplot for all features.

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

import pandas as pd

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df.hist()

plt.show()

plt.boxplot(df.values)

plt.show()
```

#9B PCA IRIS

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. Load Dataset
iris = load_iris()
X, y = iris.data, iris.target

# 2. Standardize Features
X_scaled = StandardScaler().fit_transform(X)

# 3. Apply PCA (Reduce to 2 Dimensions)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 4. Visualize the 2D Projected Data
plt.figure(figsize=(7, 5))
for i, name in enumerate(iris.target_names):
    plt.scatter(
        X_pca[y == i, 0], X_pca[y == i, 1], label=name, alpha=0.8, edgecolors="k"
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on Iris Dataset")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```

#10SET A HEAD().STATUSTS

```
import pandas as pd

data = [85, 90, 78, 92, 88]

series = pd.Series(data, name="Marks")

print("Pandas Series:")

print(series)

data_dict = {

  "Name": ["Asha", "Rahul", "Priya", "Kiran", "Sneha"],

  "Age": [20, 21, 19, 22, 20],
  "Marks": [85, 90, 78, 92, 88]
}
df = pd.DataFrame(data_dict)
print("\nPandas DataFrame:")
print(df)
print("\nFirst 5 Rows using head():")
print(df.head())
print("\nInformation about DataFrame using info():")
df.info()
print("\nStatistical Summary using describe():")
print(df.describe())
```

#10B DT

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.model_selection import train_test_split

from sklearn import metrics

data = {

  'outlook':
['Sunny','Sunny','Overcast','Rainy','Rainy','Rainy','Overcast','Sunny','Sunny','Rainy','Sunny','Ov
ercast','Overcast','Rainy'],

  'temp':
['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild','Mild','Mild','Hot','Mild'],

  'humidity':
['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal','High
','Normal','High'],

  'wind':
['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Weak','Weak','Strong','Strong'
,'Weak','Strong'],
```

```
  'play': ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No']

}

df = pd.DataFrame(data)

le = LabelEncoder()

for col in df.columns:

  df[col] = le.fit_transform(df[col])

X = df.drop('play', axis=1)

y = df['play']

X_train, X_test, y_train, y_test = train_test_split(

  X, y, test_size=0.2, random_state=0

)

model = DecisionTreeClassifier(criterion="gini", random_state=0)

model.fit(X_train, y_train)

plt.figure(figsize=(8,6))
```

```
plot_tree(model,
          feature_names=X.columns,
          class_names=["No", "Yes"],
          filled=True)
plt.title("Decision Tree using Gini Index")
plt.show()
y_pred = model.predict(X_test)
print("\nModel Accuracy:", accuracy_score(y_test, y_pred))



#SET 11 A 3X3

#set 8 3x3  Write a Python program using NumPy to create a 3×3 array and extract the first two

import numpy as np
arr = np.array([[1, 2, 3],
                [4, 5, 6],
```

```
            [7, 8, 9]])

print("Original 3x3 Array:")

print(arr)

first_two_rows = arr[0:2, :]

print("\nFirst Two Rows:")

print(first_two_rows)

first_two_columns = arr[:, 0:2]

print("\nFirst Two Columns:")

print(first_two_columns)

last_row = arr[-1, :]

print("\nLast Row:")

print(last_row
```

#11B

```
#8B lda syntethis data two class

import numpy as np

import matplotlib.pyplot as plt

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2,

                    n_classes=2, n_redundant=0,

                    random_state=0)

lda = LinearDiscriminantAnalysis(n_components=1)

X_lda = lda.fit_transform(X, y)
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Original Data")
plt.subplot(1,2,2)
plt.scatter(X_lda, [0]*len(X_lda), c=y)
plt.title("Projected Data (LDA)")
plt.yticks([])
plt.tight_layout()
plt.show()
```

```
#SET12 DETECTING OUTLIERS IN NUMERICAL DATA

import matplotlib.pyplot as plt

import pandas as pd

data = [10, 12, 15, 18, 20, 22, 25, 30, 100]

df = pd.DataFrame(data, columns=["Values"])

plt.boxplot(df["Values"])

plt.title("Boxplot for Outlier Detection")

plt.ylabel("Values")

plt.show()



#12B PCA

#5b

#Dataset: X = [[2.5, 2.4], [1.5, 1.9], [3.5, 3.0], [2.0, 2.1]]

import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

X = np.array([[2.5,2.4],
```

```
            [1.5,1.9],

            [3.5,3.0],

            [2.0,2.1]])

pca = PCA(n_components=1)

X_reduced = pca.fit_transform(X)

print("Projected Data:\n", X_reduced)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)

plt.scatter(X[:,0], X[:,1])

plt.title("Original 2D Data")
plt.subplot(1,2,2)
plt.scatter(X_reduced, [0]*len(X_reduced))
plt.title("Projected 1D Data")
plt.yticks([])
plt.tight_layout()
plt.show()
```

```
#set13 numpy for slicinbg and booleanindexing
import numpy as np

a = np.array([1, 2, 3, 4, 5, 6])

b = a.reshape(2, 3)

print("Reshaped:\n", b)

print("First Row:", b[0])

print("First 2 colums:", b[:,:2])

print("Greater than 3:", a[a > 3])



#13B NEWSAMPLE DT
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree

data = {

  'outlook': [2,2,0,1,1,1,0,2,2,1,2,0,0,1],

  'temp': [1,1,1,2,0,0,0,2,0,2,2,2,1,2],
```

```
  'humidity': [0,0,0,0,1,1,1,0,1,1,1,0,1,0],

  'wind': [1,0,1,1,1,0,0,1,1,1,0,0,1,0],

  'play': [0,0,1,1,1,0,1,0,1,1,1,1,1,0]

}

df = pd.DataFrame(data)

X = df.drop('play', axis=1)

y = df['play']

model = DecisionTreeClassifier(criterion="entropy")

model.fit(X, y)
plt.figure(figsize=(8,6))
plot_tree(model,
      feature_names=X.columns,
      class_names=["No", "Yes"],
      filled=True)
plt.title("Decision Tree - Play Tennis")
plt.show()
new_sample = [[2, 2, 0, 1]]
prediction = model.predict(new_sample)
print("Predicted Class (0=No, 1=Yes):", prediction[0])
```

```
#SET 15 A 2X2 HIST
import pandas as pd

import matplotlib.pyplot as plt

df = pd.DataFrame({

  'A': [1,2,3,4,5,6,7,8],

  'B': [2,3,4,5,6,7,8,9],

  'C': [5,6,7,8,9,10,11,12],

  'D': [10,20,30,40,50,60,70,80]

})

df.hist()

plt.tight_layout()

plt.show()




#15 B SAME DATASET PCA
```

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

df = pd.DataFrame({

  'A': [1,2,3,4,5,6,7,8],

  'B': [2,3,4,5,6,7,8,9],

  'C': [5,6,7,8,9,10,11,12],

  'D': [10,20,30,40,50,60,70,80]

})

pca = PCA(n_components=1)

X_pca = pca.fit_transform(df)
print("Projected Data (1D):\n", X_pca)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
plt.scatter(X_pca, [0]*len(X_pca))
plt.xlabel("Principal Component 1")
plt.yticks([])
plt.title("PCA Result (1D) - Without Scaling")
plt.show()
```

```
#SET16 LE ON CATEGORICAL ATTRITUBUTES

import pandas as pd

from sklearn.preprocessing import LabelEncoder

data = {

  'Name': ['Asha', 'Rahul', 'Priya', 'Kiran'],

  'Gender': ['Female', 'Male', 'Female', 'Male'],

  'Department': ['IT', 'HR', 'Finance', 'IT']

}

df = pd.DataFrame(data)

print("Original Data:")

print(df)
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
df['Department'] = le.fit_transform(df['Department'])
print("\nAfter Label Encoding:")
print(df)
```

```
#16B
#8B lda syntethis data two class

import numpy as np

import matplotlib.pyplot as plt

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2,

                    n_classes=2, n_redundant=0,

                    random_state=0)

lda = LinearDiscriminantAnalysis(n_components=1)
X_lda = lda.fit_transform(X, y)
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Original Data")
plt.subplot(1,2,2)
plt.scatter(X_lda, [0]*len(X_lda), c=y)
plt.title("Projected Data (LDA)")
plt.yticks([])
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Training Data
train_texts = [
    "Win money now",
    "Claim free prize",
    "Limited time offer",
    "Exclusive deal just for you",
    "Hello how are you",
    "Lets meet tomorrow",
    "Are you available for a call",
    "Good morning have a nice day"
]

train_labels = [1, 1, 1, 1, 0, 0, 0, 0]

# Test Data
test_texts = [
    "Earn money fast from home",
    "Urgent offer claim your prize",
    "Hey friend let us meet up",
    "Good morning call me later"
]

test_labels = [1, 1, 0, 0]
```

```
# Convert Text to Numerical Features
vectorizer = CountVectorizer()

X_train = vectorizer.fit_transform(train_texts)
X_test = vectorizer.transform(test_texts)
# Naive Bayes Model
model = MultinomialNB()
# Training
model.fit(X_train, train_labels)
# Prediction
y_pred = model.predict(X_test)
print("Predictions:")
for text, pred in zip(test_texts, y_pred):
    if pred == 1:
        print(text, "-> Spam")
    else:
        print(text, "-> Not Spam")
# Confusion Matrix
cm = confusion_matrix(test_labels, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Spam", "Spam"]
)
disp.plot()
plt.title("Confusion Matrix")
plt.show()
```